CS479: Machine Learning for 3D Data

3D Segmentation Competition

Mingue Park

Spring 2026
KAIST

Find Nubzukis!

TL;DR: Your goal is to find Nubzukis in a given scene.

Goal

- You will implement and train your own **instance segmentation model for 3D point clouds**.

- The goal is to achieve high-quality point-wise instance segmentation of a target object.

- You are encouraged to explore advanced techniques for instance segmentation.

MultiScan Dataset

The dataset consists of 3D point clouds collected from 117 indoor scenes.

MultiScan Dataset

- We use the MultiScan benchmark for the object instance segmentation task.

- You are only allowed to use the training and validation splits for training. The test split will be used for evaluation.

Finding Nubzukis in 3D Scenes

Your goal is to find target objects in a given scene. We provide a reference mesh file (`sample.glb`).

Test Dataset Generation

We will randomly generate the test data by placing multiple meshes into scenes from the test split and extracting point clouds from each mesh.

- A random number of objects will be inserted (**min = 1, max = 5**).
- Meshes are placed using multiple scale ratios, ranging from 0.025 to 0.2 of the scene diagonal.
- An object may be placed on top of another object based on its bounding box alignment, with partial overhang allowed.

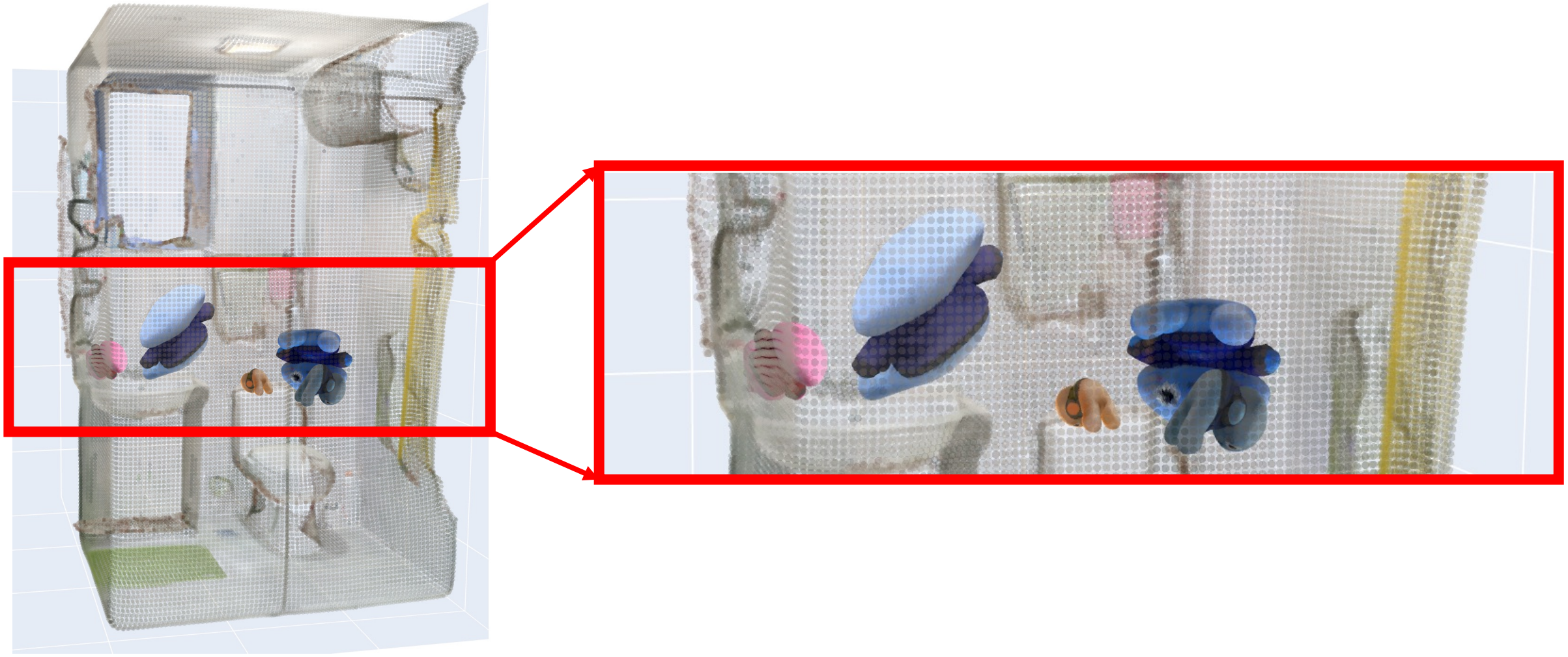
Test Dataset Generation

The mesh will be placed using the following augmentations:

- **Anisotropic scaling**: Each of the x-, y-, and z-axes is independently scaled within the range (0.5, 1.5).
- **Affine transform**: The object is rotated around the x-, y-, and z-axes within the range (-180, 180).
- **Color map jittering**

We encourage minimizing overlap between inserted objects and th the original scene.

Test Dataset Example



Test Dataset Format

Each test scene is saved as a `.npy` file with the following fields:

- `xyz: float32, shape=(N, 3)`
- `rgb: uint8, shape=(N, 3)`
- `normal: float32, shape=(N, 3)`
- `instance_labels: int32, shape=(N,)`
  * (0 for background, positive IDs for inserted instances)

Output Format

The provided evaluator will automatically save the output for each test scene as a `.npy` file in the following format:

- `predicted_label` ($\hat{y}$): `shape = (N,)`, $\hat{y}_i \in \{0,1,2,\ldots\}$

Codebase Structure

Codebase Structure

```
CS479-Assignment-3DPC-Segmentation-Challenge/
├── assets/
│   ├── sample.glb          # Object mesh data
│   └── test_0000.npy       # Test data example
│
├── dataset.py              # Code for loading test dataset (provided, DO NOT MODIFY)
├── evaluate.py             # Evaluation script (provided, DO NOT MODIFY)
├── model.py                # Model definition (students SHOULD modify)
└── visualize.py            # Visualization script (provided, CAN modify)
```

- **DO NOT modify**: Files that must be kept as-is (for fair comparison)
- **SHOULD modify**: Main files for implementing your solution
- **CAN modify**: Optional files

Codebase Structure

```
CS479-Assignment-3DPC-Segmentation-Challenge/
├── assets/
│   ├── sample.glb          # Object mesh data
│   └── test_0000.npy       # Test data example
│
├── dataset.py              # Code for loading test dataset (provided, DO NOT MODIFY)
├── evaluate.py             # Evaluation script (provided, DO NOT MODIFY)
├── model.py                # Model definition (students SHOULD modify)
└── visualize.py            # Visualization script (provided, CAN modify)
```

Keep them fixed

- **DO NOT modify**: Files that must be kept as-is (for fair comparison)
- **SHOULD modify**: Main files for implementing your solution
- **CAN modify**: Optional files

Codebase Structure

```
CS479-Assignment-3DPC-Segmentation-Challenge/
├── assets/
│   ├── sample.glb          # Object mesh data
│   └── test_0000.npy       # Test data example
│
├── dataset.py              # Code for loading test dataset (provided, DO NOT MODIFY)
├── evaluate.py             # Evaluation script (provided, DO NOT MODIFY)
├── model.py                # Model definition (students SHOULD modify)
└── visualize.py            # Visualization script (provided, CAN modify)
```

Main part to implement

- **DO NOT modify**: Files that must be kept as-is (for fair comparison)
- **SHOULD modify**: Main files for implementing your solution
- **CAN modify**: Optional files

What to Do

What to Do

- We provide the base scene as a 3D point cloud (MultiScan benchmark) and a reference object (Nubzuki mesh file).
- Your goal is to **design and implement your own point cloud instance segmentation pipeline**, including:
  - **model architecture design**,
  - **training data generation**, and
  - **instance segmentation post-processing**.

Implement Your Own Model

- You may modify the provided code (`model.py`) and add new files if necessary.
- You must implement the following two functions used by our evaluation code.

```python
def initialize_model(
    ckpt_path: str,
    device: torch.device,
    in_channels: int = 9,
    num_classes: int = 2,
) -> nn.Module:
```

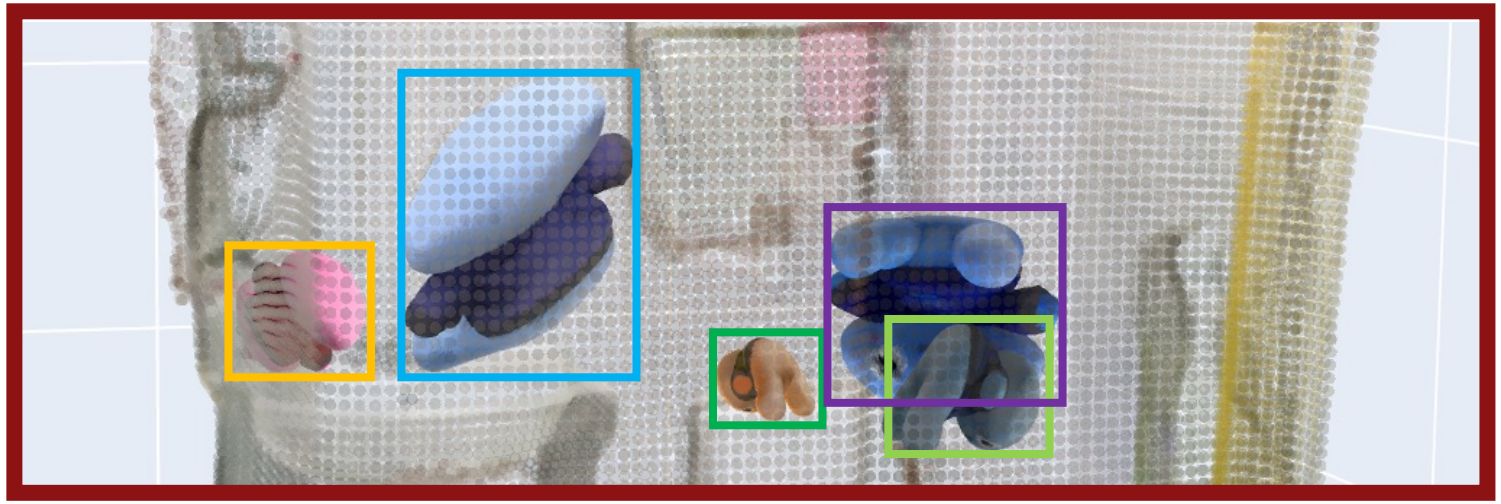
```python
def run_inference(
    model: nn.Module,
    features: torch.Tensor,
    **kwargs
) -> torch.Tensor:
```

Train the Model

- You may freely use your own training pipeline, including model training and dataset preparation.
- You may use the provided mesh and the MultiScan dataset, but you must not use the test split for training.

Test the Model

- Your code will be evaluated using the provided script (`evaluate.py`).
- You may create your own test dataset for local testing, but the final evaluation will be performed on our test dataset.



Evaluation

Evaluation

The output will be evaluated on the generated test set as follows:

- Ground-truth objects: $\{GT^1\}_1^{N_1}, \{GT^2\}_1^{N_2}, \ldots, \{GT^i\}_1^{N_i}$
- Predicted objects: $\{Pred^1\}_1^{M_1}, \{Pred^2\}_1^{M_2}, \ldots, \{Pred^j\}_1^{M_j}$
- We perform **one-to-one matching** between ground-truth and predicted instances using the Hungarian algorithm with 1 - IoU as the cost function.

Evaluation

- TP_τ: Number of matched instance pairs whose IoU is at least τ.
- FP_τ: Number of predicted instances not counted as true positives.
- FN_τ: Number of Ground-truth instances not counted as true positives.

Evaluation

- TP_τ: Number of matched instance pairs whose IoU is at least τ.
- FP_τ: Number of predicted instances not counted as true positives.
- FN_τ: Number of Ground-truth instances not counted as true positives.

Based on this, we define the $F1_\tau$ score as follows:

$$F1_\tau = \frac{2 * TP_\tau}{2 * TP_\tau + FP_\tau + FN_\tau}$$

Evaluation

- We will evaluate F1@0.25 ($F1_{0.25}$) and F1@0.5 ($F1_{0.5}$).
- Final grading will be determined relative to the best score achieved for each metric:

$$\text{Score} = \max\left(\frac{\text{Your Score}}{\text{Highest Score}} \times 8, 0\right)$$

Evaluation

- You can earn bonus credit for each metric:
  - **Mid-term evaluation bonus**: The top-*k* teams for each metric in the mid-term evaluation will receive +1.
  - **Winner bonus**: If your team achieves the highest score for each metric, you will receive +1.

- In total, the 3D segmentation challenge is worth a maximum of 20 points.

What to Submit

Overview of Submission

- There are two submission deadlines:
  the mid-term evaluation (*optional*) and **the final submission**.

- The mid-term evaluation is optional, but top-*k* teams for each metrics will earn bonus credit.

- The purpose of the mid-term evaluation is to check your team's standing relative to other teams.

Mid-Term Evaluation Submission

- The scores (F1@0.25, F1@0.5) will be published on the leaderboard.

- The top-*k* submissions will earn bonus credit.

Mid-Term Evaluation Submission

- **Due**: April 30 (Thursday) 23:59 KST.
- **What to Submit**:
  - **Self-contained source code**
    - Your submission must include the complete codebase.
    - TAs will run your code in their environment without any modifications.
  - **Model checkpoint and configs, if applicable**
  - TAs will run the evaluation script from the root directory of your submission using the following command:

    *Please save your model checkpoint in (./checkpoints/best.pth).

```
python evaluate.py --ckpt-path ./checkpoints/best.pth
```

Final Submission

- **Due**: May 9 (Saturday), 23:59 KST.
- **What to Submit**:
  - Same as the mid-term evaluation submission, **plus a 2-page write-up**.
- **Write-up Guidelines:**
  - No template is required. Maximum of two A4 pages, excluding references.
  - Contents should include:
    - **Technical details**: A one-paragraph description.
    - **Training details**: Training logs and total training time.
    - **Qualitative evidence**: Sample rendered images with segmentation results.
    - **Citations:** All external code and papers used.

Self-Evaluation Checklist

Before submitting:

- Use the TA's environment setup for evaluation.
- Code runs end-to-end: training and evaluation without errors.
- Correct checkpoint path: `./checkpoints/best.pth`.
- Citation included: all external code and papers in the final write-up.

Grading

Please note that your score may be reduced according to the following rules:

- **Late submission**: Zero score.
- **Missing any required item in the final submission**: Zero score.
- **Missing items in the write-up**: 10% penalty for each.

Q&A

- Further details are provided in the README.md.
- Any questions?